Imprimir / Guardar PDF

PROGRAMACIÓN · DAW 1.º · CURSO 2024-2025

PROYECTO INTEGRADO FINAL · RA5 · RA6 · RA7

RA5 RA6 RA7

Proyecto Integrado

POO avanzada · Tipos de datos · E/S y GUI

Trabajo en **parejas** · Duración: 3-4 semanas

Entrega mediante repositorio GitHub con tags por hito
Presentación oral al resto de la clase

Índice

1. Descripción general
2. Hitos de entrega
3. Elección del proyecto
4. Requisitos técnicos mínimos
5. Presentación oral
6. Rúbrica RA7 — POO avanzada
7. Rúbrica RA6 — Tipos avanzados
8. Rúbrica RA5 — E/S y GUI
9. Rúbrica presentación oral
10. Checklist por hito
11. Documentación y recursos

1. Descripción general

Desarrollaréis en parejas una **aplicación Java completa** sobre un dominio de vuestra elección. La aplicación debe tener interfaz de consola e interfaz gráfica con Swing, persistir datos en ficheros CSV y exportar a JSON, y estar construida sobre una jerarquía de clases con herencia e interfaces.

El proyecto se entrega en **tres hitos**, cada uno como un **tag en el repositorio GitHub**. La nota de cada RA se evalúa de forma independiente según la rúbrica correspondiente. El proyecto finaliza con una **presentación oral de 10 minutos** al resto de la clase.

1.1 Normas generales

✗ **NO se permite**

✗ Librerías externas en el `pom.xml` (solo Java SE puro).

✗ Editor visual de GUI (NetBeans Matisse, IntelliJ GUI Designer). La GUI se construye a mano.

✗ Bases de datos. La persistencia es exclusivamente mediante ficheros.

✗ Serialización con `ObjectOutputStream`. El formato es CSV y JSON textual.

✗ Entregas en .zip. Solo se acepta la URL del repositorio GitHub.

✗ Código copiado de otro grupo. Se comprobará similitud entre entregas.

✓ **SÍ se permite y recomienda**

✓ Usar Maven únicamente como herramienta de build, sin añadir dependencias.

✓ Consultar la documentación oficial de Java y los fragmentos de este enunciado.

✓ Hacer commits frecuentes que muestren la evolución real del trabajo.

✓ Proponer un dominio propio (con validación previa del profesor, ver sección 3.2).

✓ Añadir funcionalidades extra una vez cubiertos todos los requisitos.

2. Hitos de entrega

El proyecto se divide en tres hitos con fecha de entrega independiente. Cada hito se entrega creando un **tag en el repositorio Git** con el nombre indicado. Un hito sin tag no se considera entregado.

Hito 1 — Modelo de clases Tag: `v1.0-modelo` Semana 1-2 RA7 principalmente	Entregáis el **diagrama UML** (imagen en el repositorio) y la implementación completa del modelo de clases: superclase abstracta, subclases, interfaces, Javadoc y método genérico. Debe compilar sin errores. Un `Main` de prueba crea objetos de cada subclase y los imprime por consola para verificar la jerarquía.
Hito 2 — Lógica y consola Tag: `v2.0-consola` Semana 2-3 RA6 RA5	La aplicación funciona completamente por consola: menú con `printf`, colecciones (`ArrayList` + `HashMap`), iteradores, streams, expresiones regulares, lectura y escritura de CSV con `BufferedReader` / `FileWriter` y `RandomAccessFile`, y exportación a JSON manual.
Hito 3 — GUI completa Tag: `v3.0-gui` Semana 3-4	Interfaz gráfica con Swing construida a mano: `JTable` con `DefaultTableModel`, `JDialog` modal para añadir/editar, buscador con filtrado en tiempo real, `WindowListener` para

RA5 f/g/h	gestionar el cierre, al menos un `ActionListener` con lambda y otro con clase anónima.

> *El historial de commits es parte de la evaluación. Un repositorio con un único commit final o con todos los commits el mismo día genera dudas sobre la autoría del trabajo. Se espera ver evolución real a lo largo de las semanas.*

2.1 Estructura del repositorio GitHub

```
nombre-proyecto/                ← nombre del repositorio
├── pom.xml
├── README.md                   ← descripción, instrucciones, nombres del equipo
├── uml/
│   └── diagrama.png            ← diagrama UML (entregado en Hito 1)
├── datos/
│   └── catalogo.csv            ← fichero de datos
└── src/main/java/com/equipo/proyecto/
    ├── modelo/                 ← clases del dominio
    ├── persistencia/           ← GestorFicheros
    ├── colecciones/            ← clase que gestiona las colecciones
    ├── consola/                ← MenuConsola
    └── gui/                    ← VentanaPrincipal, DialogoFormulario
```

3. Elección del proyecto

Podéis elegir uno de los ocho proyectos propuestos a continuación, o proponer uno propio (ver sección 3.2). Todos comparten los mismos requisitos técnicos y se evalúan con las mismas rúbricas.

3.1 Proyectos propuestos

🎮 GameVault

Gestión de colección de videojuegos

Jerarquía: `Videojuego` → `JuegoFisico` / `JuegoDigital`

Interfaces: `Valorable` , `Exportable`

Colecciones: `ArrayList` catálogo + `HashMap` por género

Perfil: Técnico / gamer

📚 BiblioTrack

Gestión de biblioteca personal

Jerarquía: `Publicacion` → `Novela` / `LibroTecnico` / `Comic`

Interfaces: `Prestable` , `Resenable`

Colecciones: `ArrayList` libros + `HashMap` por autor

Perfil: Tranquilo, dominio sencillo

🏋️ FitLog

Registro de entrenamientos

🎵 SoundBox

Gestión de discografía personal

Jerarquía: `Ejercicio` → `Cardio` / `Fuerza` / `Flexibilidad`

Interfaces: `Medible` , `Exportable`

Colecciones: `ArrayList` sesiones + `HashMap` por semana

Perfil: Deportivo, cálculos numéricos

Jerarquía: `Album` → `AlbumFisico` / `AlbumDigital`

Interfaces: `Reproducible` , `Puntuable`

Colecciones: `ArrayList` álbumes + `HashMap` por artista

Perfil: Musical

🍽️ RecipeBox

Gestor de recetas de cocina

Jerarquía: `Receta` → `RecetaVegana` / `RecetaRapida` / `RecetaSinGluten`

Interfaces: `Planificable` , `Nutricional`

Colecciones: `ArrayList` recetas + `HashMap` por categoría

Perfil: Creativo / diseño

🌍 TravelLog

Diario de viajes

Jerarquía: `Viaje` → `ViajeNacional` / `ViajeInternacional`

Interfaces: `Presupuestable` , `Compartible`

Colecciones: `ArrayList` viajes + `HashMap` por país

Perfil: Viajero

💼 TaskFlow

Gestor de tareas y proyectos

Jerarquía: `Tarea` → `TareaSimple` / `TareaConSubtareas` / `TareaRecurrente`

Interfaces: `Priorizable` , `Notificable`

Colecciones: `ArrayList` tareas + `HashMap` por estado

Perfil: Empresarial / gestión

🐾 PetCare

Gestión de mascotas y cuidados

Jerarquía: `Mascota` → `Perro` / `Gato` / `Ave`

Interfaces: `Cuidable` , `Registrable`

Colecciones: `ArrayList` mascotas + `HashMap` por especie

Perfil: Accesible, dominio cercano

> **Todos los proyectos tienen la misma complejidad técnica requerida.** La elección debe basarse en el dominio que más os motive. Los requisitos de herencia, colecciones, streams y GUI son idénticos para todos.

3.2 Propuesta propia

Si queréis trabajar sobre un dominio diferente, rellenad la siguiente **ficha de validación** y entregadla al profesor **antes del inicio del Hito 1**. Sin validación aprobada no se puede empezar con un proyecto propio.

Campo	Vuestra propuesta
Nombre del proyecto	
Descripción (3 líneas máx.)	
Superclase abstracta	
Subclase 1 + atributos propios	

Campo	Vuestra propuesta
Subclase 2 + atributos propios	
Subclase 3 (opcional)	
Interfaz 1 + método que define	
Interfaz 2 + método que define	
Colección principal (ArrayList<?>)	
Colección secundaria (HashMap<?,?>)	
Qué exportaréis a JSON	
Aprobado por el profesor	☐ Sí ☐ No Fecha: _____

4. Requisitos técnicos mínimos

Independientemente del dominio elegido, todos los proyectos deben cumplir los siguientes requisitos. Son los que se verificarán en la evaluación.

RA7 — Modelo de clases

- ✓ Una **superclase abstracta** con al menos 2 métodos abstractos y validaciones en el constructor.
- ✓ Al menos **2 subclases concretas** que usen `super()` y sobrescriban los métodos abstractos.
- ✓ Al menos **2 interfaces** con propósito diferenciado, implementadas por la jerarquía.
- ✓ Un **método genérico** con bounded type (`<T extends SuperClase>`).
- ✓ **Javadoc** en todas las clases y métodos públicos (@param, @return, @throws donde aplique).
- ✓ Un diagrama UML en la carpeta `uml/` del repositorio.

RA6 — Tipos avanzados

- ✓ Un **array** como atributo en alguna clase del modelo (p.ej. categorías, etiquetas).
- ✓ **ArrayList** como colección principal de objetos del dominio.
- ✓ **HashMap** como índice secundario para búsquedas por clave.
- ✓ Un **Iterator explícito** en el método de eliminación.
- ✓ Al menos **4 operaciones con Stream**: filter, sorted, groupingBy y una operación de reducción.

- ✓ **Expresiones regulares** en al menos 2 sitios: una búsqueda y una validación.
- ✓ **Exportación a JSON** construida manualmente con StringBuilder. El JSON debe ser válido.

RA5 — E/S y GUI

- ✓ Menú de consola con formato tabular usando `printf`.
- ✓ **3 mecanismos de E/S de ficheros**: BufferedReader (lectura), FileWriter+BufferedWriter (escritura), RandomAccessFile (acceso por ID).
- ✓ Todos los accesos a fichero con **try-with-resources**.
- ✓ GUI Swing construida **a mano**: JTable + JScrollPane, JDialog modal, JTextField buscador.
- ✓ Al menos un **ActionListener con lambda** y otro con **clase anónima**.
- ✓ **WindowListener** que intercepta el cierre y pregunta si guardar.

5. Presentación oral

Cada pareja realizará una **presentación de 10 minutos** al resto de la clase al finalizar el proyecto. La presentación se hace con el proyecto en pantalla e incluye obligatoriamente las tres partes siguientes:

Parte	Duración	Contenido
Demo en vivo	~4 min	Ejecutáis la aplicación y recorréis los casos de uso principales: añadir, buscar, guardar, cerrar y reabrir comprobando que los datos persisten.
Decisiones de diseño	~4 min	Explicáis vuestra jerarquía de clases (con el diagrama UML), por qué elegisteis cada interfaz y qué colección usasteis para cada propósito.
Lecciones aprendidas	~2 min	Qué fue lo más difícil, qué refactorizaríais si empezarais de nuevo, qué herramienta o concepto os resultó más útil.

> Ambos miembros de la pareja deben participar activamente en la presentación. El profesor puede hacer preguntas a cualquiera de los dos sobre cualquier parte del código.

6. Rúbrica RA7 — POO avanzada

Se evalúa independientemente del dominio elegido. Los indicadores describen el nivel de dominio conceptual, no una implementación concreta.

Criterio	Insuficiente (1–4)	Suficiente (5–6)	Notable (7–8)	Sobresaliente (9–10)
a-e Jerarquía de herencia	No hay herencia real o es puramente cosmética (subclase sin añadir nada).	Superclase + 2 subclases con @Override. Los constructores usan super().	Superclase abstracta con métodos abstractos. Polimorfismo usado en colecciones.	Jerarquía bien justificada en la presentación. Se explotan instanceof y casting con sentido.
f-g Prueba y funcionamiento	El código no compila o la jerarquía produce errores en ejecución.	Compila y funciona, pero hay bugs evidentes en el comportamiento polimórfico.	Jerarquía probada con objetos de cada subclase. Los métodos abstractos funcionan correctamente.	Casos límite gestionados. Excepciones propias o validaciones en constructores. Explicado en presentación.
h Documentación Javadoc	Sin comentarios o solo comentarios inline sin estructura.	Javadoc presente en las clases principales pero incompleto.	Javadoc en todas las clases y métodos públicos con @param y @return.	Javadoc completo con @throws, @see y ejemplos donde añaden valor. Generado con `mvn javadoc:javadoc`.
i-j Interfaces y composición	No hay interfaces o la única interfaz no tiene sentido de contrato.	Una interfaz implementada correctamente por alguna clase.	Dos interfaces con propósito diferenciado. La pareja sabe explicar qué contrato define cada una.	Explican en la presentación cuándo eligieron interfaz vs herencia y por qué. Composición usada conscientemente.

> *El criterio h (Javadoc) es obligatorio para todos los proyectos. Clases públicas sin Javadoc suponen automáticamente un Insuficiente en ese criterio.*

7. Rúbrica RA6 — Tipos avanzados de datos

Se evalúa independientemente del dominio. La pareja debe ser capaz de justificar en la presentación por qué eligió cada estructura de datos para cada propósito.

Criterio	Insuficiente (1–4)	Suficiente (5–6)	Notable (7–8)	Sobresaliente (9–10)
a Arrays	No se usan arrays en ningún lugar del modelo.	Un array declarado como atributo en alguna clase.	Array usado con recorrido y manipulación (búsqueda, copia, transformación).	Uso justificado de array frente a lista. La pareja explica cuándo prefiere array.
c-e Colecciones e iteradores	Solo se usa un tipo de colección o no se usan colecciones del framework.	ArrayList usado correctamente como colección principal.	ArrayList + HashMap con propósito diferenciado. Iterator explícito en eliminación.	La pareja explica en presentación por qué eligió cada colección y sus trade-offs.
f Genéricos	No hay ningún método genérico.	Un método genérico presente aunque sea básico.	Método genérico con bounded type (`<T extends …>`) usado con sentido real.	Varios métodos genéricos. La pareja explica el concepto de type bound en la presentación.
g Expresiones regulares	No se usan expresiones regulares.	Regex usada en una validación simple.	Regex en búsqueda de patrones + validación. Pattern compilado y reutilizado.	Flags (CASE_INSENSITIVE), grupos de captura o alternancia. Explicados en presentación.
h-i JSON manual	No hay exportación o se usa una librería externa.	Exportación a JSON básica con campos simples.	JSON válido con arrays anidados. Caracteres especiales escapados correctamente.	JSON válido verificado con jsonlint.com. Importación desde JSON también implementada.
k Stream API	No se usa Stream API o solo `.stream().collect()` sin operaciones intermedias.	`filter()` o `map()` usados en al menos un método.	filter + sorted + collect. groupingBy o averagingDouble en al menos un método.	4 o más operaciones distintas. Method references (`::`). Explicadas en presentación.

> Los streams y las expresiones regulares son los dos conceptos que más aparecen generados por IA sin comprensión real. En la presentación se preguntará específicamente sobre ellos.

8. Rúbrica RA5 — Entrada/Salida e interfaces gráficas

Se evalúa la correcta aplicación de los mecanismos de E/S del lenguaje. Los criterios f, g y h (GUI) se evalúan en el Hito 3.

Criterio	Insuficiente (1–4)	Suficiente (5–6)	Notable (7–8)	Sobresaliente (9–10)
a-b Consola y formatos	No hay menú o la salida es texto plano sin formato.	Menú funcional con opciones numeradas. Salida legible.	printf con especificadores de anchura para tabla alineada. Separadores visuales.	Formato pulido y consistente. La pareja puede explicar cada especificador de printf usado.
c Posibilidades E/S	Solo se usa un mecanismo de E/S.	Scanner para consola + un tipo de acceso a fichero.	Scanner + BufferedReader + FileWriter + RandomAccessFile. Cada uno en su contexto correcto.	La pareja justifica en presentación por qué eligió cada clase de E/S para cada operación.
d-e Ficheros y métodos de acceso	No hay persistencia real o los datos se pierden al cerrar.	Lectura y escritura de CSV funcionan correctamente.	3 mecanismos: lectura secuencial, escritura y acceso por posición con RandomAccessFile.	try-with-resources en todos los accesos. Gestión de fichero inexistente. Explicado en presentación.
f-h GUI Swing	No hay GUI, la GUI falla al arrancar, o se detecta código de editor visual.	Ventana con componentes básicos. JTable muestra datos. Los botones responden.	JTable con DefaultTableModel. JDialog modal. Buscador funcional. WindowListener implementado.	Lambda + clase anónima explicadas. Validaciones en JDialog. UX cuidada. Demo fluida en presentación.

> *Si se detecta código generado por editor visual (método `initComponents()` masivo, variables `jButton1`, `jLabel2` ...) el criterio f-h se califica automáticamente con 0.*

9. Rúbrica de la presentación oral

La presentación se evalúa de forma transversal e independiente de los tres RA. Ambos miembros de la pareja comparten la misma nota de presentación.

Criterio	Insuficiente (1–4)	Suficiente (5–6)	Notable (7–8)	Sobresaliente (9–10)
Demo en vivo	La aplicación no arranca o falla en los casos básicos.	Demo sin errores graves. Cubre los casos de uso principales.	Demo fluida. Muestra persistencia, búsqueda y GUI sin incidentes.	Demuestra casos límite, gestión de errores y funcionalidades extra.
Dominio del código	No pueden explicar qué hace su propio código cuando se les pregunta.	Explican la estructura general del proyecto.	Explican decisiones de diseño concretas: por qué este HashMap, este stream, esta interfaz.	Justifican alternativas que descartaron y por qué. Responden preguntas del resto de la clase.
Claridad y estructura	Exposición confusa, sin hilo claro o muy por debajo de los 8 minutos.	Cubre las tres partes mínimas. Ambos miembros participan.	Bien estructurada, ritmo adecuado, transiciones claras entre partes.	Engancha a la audiencia. Uso eficaz del diagrama UML. Tiempo bien gestionado.

10. Checklist antes de entregar cada hito

Comprobad cada punto antes de crear el tag en GitHub. Un punto sin marcar es un requisito que falta y que se reflejará en la nota.

v1.0-modelo — Hito 1

- [ ] **RA7** La superclase es abstracta y tiene al menos 2 métodos abstractos.
- [ ] **RA7** Las subclases usan `super()` en el constructor y sobrescriben los métodos abstractos.
- [ ] **RA7** Hay al menos 2 interfaces con propósito diferenciado implementadas por la jerarquía.
- [ ] **RA7** Existe un método genérico con bounded type `<T extends SuperClase>`.
- [ ] **RA7** Todas las clases y métodos públicos tienen Javadoc con @param y @return.
- [ ] **RA6** La superclase tiene al menos un atributo de tipo array con getter.
- [ ] **RA7** El diagrama UML está en la carpeta `uml/` del repositorio.
- [ ] **GIT** El repositorio es público y el tag `v1.0-modelo` apunta al commit correcto.

v2.0-consola — Hito 2

- [ ] RA5 El menú de consola usa `printf` con columnas alineadas.
- [ ] RA5 La opción de añadir valida todos los campos con regex y bucle.
- [ ] RA5 GestorFicheros tiene: `cargar()` con BufferedReader, `guardar()` con FileWriter, `buscarPorId(`
- [ ] RA5 Todos los accesos a fichero usan `try-with-resources` .
- [ ] RA6 La colección principal es `ArrayList<SuperClase>` (polimórfica).
- [ ] RA6 Existe un `HashMap<String, List<SuperClase>>` como índice secundario.
- [ ] RA6 El método de eliminación usa `Iterator` explícito.
- [ ] RA6 Hay al menos 4 métodos que usan Stream API (filter, sorted, groupingBy, reducción).
- [ ] RA6 Las expresiones regulares se usan en al menos 1 búsqueda y 1 validación.
- [ ] RA6 El método `toJson()` genera JSON válido (verificado en jsonlint.com).
- [ ] GIT El tag `v2.0-consola` apunta al commit correcto.

v3.0-gui — Hito 3

- [ ] RA5 VentanaPrincipal construida a mano, sin editor visual.
- [ ] RA5 JTable usa DefaultTableModel y muestra todos los objetos al arrancar.
- [ ] RA5 El buscador filtra la tabla al pulsar Enter o el botón.
- [ ] RA5 El botón Añadir/Nuevo abre un JDialog modal con validación de campos.
- [ ] RA5 El botón Eliminar/Borrar pide confirmación con JOptionPane.
- [ ] RA5 Hay al menos un ActionListener con lambda y otro con clase anónima.
- [ ] RA5 WindowListener intercepta el cierre y pregunta si guardar.
- [ ] GIT El tag `v3.0-gui` apunta al commit correcto.
- [ ] GIT El README.md tiene instrucciones para compilar (`mvn package`) y ejecutar.

11. Documentación y recursos

Java SE 17 — Documentación oficial

→ JavaDoc completo Java SE 17

→ Tutorial oficial Java — All Trails

→ java.util — ArrayList, HashMap, Iterator

→ Stream API — java.util.stream

→ Expresiones regulares — java.util.regex

Java Swing

→ Tutorial oficial de Swing (Oracle)

→ Componentes Swing

→ Layout managers

→ JTable — cómo usarla

→ Eventos y listeners

Ficheros en Java

→ Basic I/O — Tutorial oficial

→ BufferedReader y BufferedWriter

Maven y GitHub

→ Guía de inicio Maven

→ Estructura estándar de directorios Maven

→ Crear un repositorio en GitHub

→ Git — Crear y usar tags

Herramientas

→ IntelliJ IDEA Community (gratis)

→ Eclipse IDE for Java Developers

→ draw.io — diagramas UML online (gratis)

→ JSONLint — validador de JSON online